
Algorithm 1 Genetic Algorithm Scheduler

Require: Task set T , job set J , core set C , cluster set K
Require: Job dependency set D_{job} , task dependency set D_{task}
Require: Eligible cores $E_t \subseteq C$ for each task $t \in T$
Require: Job release times r_j , durations p_j , deadlines d_j
Require: Task memory demands m_t
Require: Core memory budgets B_c^{core} , core WCET scales α_c
Require: Cluster memory budgets B_k^{cluster}
Require: Communication penalties and penalty weights
Require: Time limit and maximum generation count
Ensure: Best decoded schedule found by the genetic algorithm

- 1: $N \leftarrow |T|$
- 2: Define chromosome length $2N$ $\triangleright$ First N genes encode task assignment, last N genes encode task priority
- 3: Define gene space:
- 4: **for** each task $t \in T$ **do**
- 5: assignment gene $a_t \in \{0, \dots, |E_t| - 1\}$ $\triangleright$ Index into eligible core list
- 6: **end for**
- 7: **for** each task $t \in T$ **do**
- 8: priority gene $\pi_t \in [0, 1]$
- 9: **end for**
- 10: Initialize GA population
- 11: **while** time limit and generation limit are not reached **do**
- 12: **for** each chromosome x in population **do**
- 13: $\phi(x) \leftarrow \text{FITNESS}(x)$
- 14: **end for**
- 15: Create next generation using GA selection, crossover, and mutation
- 16: **end while**
- 17: $x^* \leftarrow$ best chromosome found
- 18: **return** $\text{DECODECHROMOSOME}(x^*)$

Algorithm 2 Fitness Evaluation

Require: Chromosome x , current generation g
Ensure: Fitness value $\phi(x)$

- 1: decoded solution $\leftarrow \text{DECODECHROMOSOME}(x, g)$
- 2: $J_{\text{total}} \leftarrow$ decoded solution total cost
- 3: $\phi(x) \leftarrow \frac{1}{1 + J_{\text{total}}}$
- 4: **return** $\phi(x)$

Algorithm 3 Decode Chromosome

Require: Chromosome x of length $2|T|$, current generation g **Ensure:** Assignment, priority order, schedule, violations, objective cost, and total cost

```
1:  $A^T \leftarrow \text{DECODETASKASSIGNMENT}(x)$   $\triangleright$  Task-to-core assignment
2:  $\Pi^T \leftarrow \text{DECODETASKPRIORITY}(x)$   $\triangleright$  Task priority values

3: for each job  $j \in J$  do
4:    $A_j^J \leftarrow A_{\tau(j)}^T$   $\triangleright$  Jobs inherit the core of their parent task
5: end for
6:  $P^J \leftarrow \text{DERIVEJOBPRIORITYORDER}(\Pi^T)$ 
7:  $(S, F) \leftarrow \text{BUILDSCHEDULE}(A^J, P^J)$ 

8:  $(O^{\text{core}}, O^{\text{cluster}}) \leftarrow \text{MEMORYOVERFLOW}(A^T)$ 
9:  $Q \leftarrow \text{COMMUNICATIONCOST}(A^T)$ 

12:  $V_{\text{prec}} \leftarrow \text{PRECEDENCEVIOLATION}(S, F)$ 
11:  $V_{\text{dead}} \leftarrow \text{TERMINALDEADLINEVIOLATION}(F)$ 
12:  $V_{\text{overlap}} \leftarrow \text{SAMECOREOVERLAPVIOLATION}(S, F, A^J)$ 
13:  $V_{\text{jitter}} \leftarrow \text{JITTERVIOLATION}(S)$ 
14:  $V \leftarrow V_{\text{prec}} + V_{\text{dead}} + V_{\text{overlap}} + V_{\text{jitter}}$ 

15:  $\gamma \leftarrow \min\left(1, \frac{g}{G_{\text{max}}}\right)$ 
16:  $w_V \leftarrow 100 + 999900 \cdot \gamma$ 
17:  $J_{\text{viol}} \leftarrow w_V \cdot V$ 

18:  $J_{\text{obj}} \leftarrow \lambda_{\text{core}} \sum_{c \in C} O_c^{\text{core}} + \lambda_{\text{cluster}} \sum_{k \in K} O_k^{\text{cluster}} + Q$ 
19:  $J_{\text{total}} \leftarrow J_{\text{obj}} + J_{\text{viol}}$ 
20: return  $(A^J, P^J, S, F, O^{\text{core}}, O^{\text{cluster}}, Q, V, J_{\text{obj}}, J_{\text{total}})$ 
```

Algorithm 4 Decode Task Assignment

Require: Chromosome x **Ensure:** Task-to-core assignment A^T

```
1: for each task  $t \in T$  do
2:    $q_t \leftarrow \text{round}(a_t)$ 
3:    $q_t \leftarrow \max(0, \min(q_t, |E_t| - 1))$   $\triangleright$  Clamp to valid eligible-core index
4:    $A_t^T \leftarrow E_t[q_t]$ 
5: end for
6: return  $A^T$ 
```

Algorithm 5 Decode Task Priority

Require: Chromosome x **Ensure:** Task priority values Π^T

- 1: **for** each task $t \in T$ **do**
 - 2: $\Pi_t^T \leftarrow \pi_t$ $\triangleright$ Priority gene of task t
 - 3: **end for**
 - 4: **return** Π^T
-

Algorithm 6 Derive Job Priority Order

Require: Task priority values Π^T **Ensure:** Job priority order P^J

- 1: $L \leftarrow []$
 - 2: **for** each job $j \in J$ **do**
 - 3: $t \leftarrow \tau(j)$
 - 4: Append $(j, \Pi_t^T, r_j, \text{index}(j))$ to L
 - 5: **end for**
 - 6: Sort L by descending Π_t^T
 - 7: Break ties by ascending release time r_j
 - 8: Break remaining ties by original job index
 - 9: $P^J \leftarrow$ ordered job IDs from L
 - 10: **return** P^J
-

Algorithm 7 Build Schedule from Job Assignment and Job Priority Order

Require: Job-to-core assignment A^J , job priority order P^J **Ensure:** Start times S_j and finish times F_j

```
1: for each core  $c \in C$  do
2:    $I_c \leftarrow []$  ▷ Scheduled intervals on core  $c$ 
3: end for
4:  $S \leftarrow \emptyset, F \leftarrow \emptyset$ 
5:  $A_{\text{anchored}} \leftarrow \emptyset$ 

6: if strict chain-root start is enabled then
7:   for each job  $j \in J$  do
8:     if  $j$  is a chain root then
9:        $c \leftarrow A_j^J$ 
10:       $S_j \leftarrow r_j$ 
11:       $F_j \leftarrow S_j + p_j \alpha_c$ 
12:      Insert interval  $(S_j, F_j, j)$  into  $I_c$ 
13:       $A_{\text{anchored}} \leftarrow A_{\text{anchored}} \cup \{j\}$ 
14:    end if
15:  end for
16: end if

17: for each job  $j \in J$  do
18:    $\rho_j \leftarrow$  position of  $j$  in  $P^J$  ▷ Lower rank means higher priority
19:    $n_j \leftarrow |\text{pred}(j)|$  ▷ Number of unscheduled predecessors
20:    $H_j \leftarrow 0$  ▷ Maximum predecessor finish time
21: end for
22: Initialize empty ready heap  $R$ 
23: for each job  $j \in J$  do
24:   if  $n_j = 0$  then
25:     Push  $(r_j, \rho_j, \text{index}(j), j)$  into  $R$ 
26:   end if
27: end for
28:  $V \leftarrow \emptyset$  ▷ Visited jobs
29: while  $R \neq \emptyset$  do
30:   Pop highest-priority ready job  $j$  from  $R$ 
31:   if  $j \in V$  then
32:     continue
33:   end if
34:    $V \leftarrow V \cup \{j\}$ 
35:   if  $j \notin A_{\text{anchored}}$  then
36:      $c \leftarrow A_j^J$ 
37:      $e_j \leftarrow \max(r_j, H_j)$ 
38:      $S_j \leftarrow \text{FIND\_EARLIEST\_GAP}(I_c, e_j, p_j \alpha_c)$ 
39:      $F_j \leftarrow S_j + p_j \alpha_c$ 
40:     Insert interval  $(S_j, F_j, j)$  into  $I_c$  in sorted order
41:   end if
42:   for each successor  $h \in \text{succ}(j)$  do
43:      $H_h \leftarrow \max(H_h, F_j)$  4
44:      $n_h \leftarrow n_h - 1$ 
45:     if  $n_h = 0$  then
46:       Push  $(\max(r_h, H_h), \rho_h, \text{index}(h), h)$  into  $R$ 
47:     end if
48:   end for
49: end while
50: if  $|V| \neq |J|$  then
51:   error "Cycle or unschedulable dependency graph"
52: end if
53: return  $(S, F)$ 
```

Algorithm 8 Find Earliest Gap on Core

Require: Sorted interval list I_c , earliest start e , duration p **Ensure:** Earliest feasible start time

```
1:  $s \leftarrow e$ 
2: for each interval  $(u, v, \cdot) \in I_c$  in ascending order do
3:   if  $s + p \leq u$  then
4:     return  $s$ 
5:   end if
6:   if  $s < v$  then
7:      $s \leftarrow v$ 
8:   end if
9: end for
10: return  $s$ 
```

Algorithm 9 Memory Overflow Penalty

Require: Task-to-core assignment A^T **Ensure:** Core overflows O^{core} and cluster overflows O^{cluster}

```
1: for each core  $c \in C$  do
2:    $M_c^{\text{core}} \leftarrow 0$ 
3: end for
4: for each cluster  $k \in K$  do
5:    $M_k^{\text{cluster}} \leftarrow 0$ 
6: end for
7: for each task  $t \in T$  do
8:    $c \leftarrow A_t^T$ 
9:    $k \leftarrow \text{cluster}(c)$ 
10:   $M_c^{\text{core}} \leftarrow M_c^{\text{core}} + m_t$ 
11:   $M_k^{\text{cluster}} \leftarrow M_k^{\text{cluster}} + m_t$ 
12: end for
13: for each core  $c \in C$  do
14:    $O_c^{\text{core}} \leftarrow \max(0, M_c^{\text{core}} - B_c^{\text{core}})$ 
15: end for
16: for each cluster  $k \in K$  do
17:    $O_k^{\text{cluster}} \leftarrow \max(0, M_k^{\text{cluster}} - B_k^{\text{cluster}})$ 
18: end for
19: return  $(O^{\text{core}}, O^{\text{cluster}})$ 
```

Algorithm 10 Communication Cost

Require: Task-to-core assignment A^T , task dependency set D_{task} **Ensure:** Communication cost Q

```
1:  $Q \leftarrow 0$ 
2: for each task dependency  $(t_1, t_2) \in D_{\text{task}}$  do
3:    $c_1 \leftarrow A_{t_1}^T$ 
4:    $c_2 \leftarrow A_{t_2}^T$ 
5:   if an explicit communication penalty exists for  $(c_1, c_2)$  then
6:      $Q \leftarrow Q + q_{c_1 c_2}^{\text{explicit}}$ 
7:   else if  $c_1 = c_2$  then
8:      $Q \leftarrow Q + w_{\text{intra}}$ 
9:   else if  $\text{cluster}(c_1) = \text{cluster}(c_2)$  then
10:     $Q \leftarrow Q + w_{\text{intercore}}$ 
11:   else
12:     $Q \leftarrow Q + w_{\text{intercluster}}$ 
13:   end if
14: end for
15: return  $Q$ 
```

Algorithm 11 Precedence Violation

Require: Start times S , finish times F , job dependency set D_{job} **Ensure:** Precedence violation V_{prec}

```
1:  $V_{\text{prec}} \leftarrow 0$ 
2: for each job dependency  $(j, h) \in D_{\text{job}}$  do
3:   if  $S_h < F_j$  then
4:      $V_{\text{prec}} \leftarrow V_{\text{prec}} + F_j - S_h$ 
5:   end if
6: end for
7: return  $V_{\text{prec}}$ 
```

Algorithm 12 Terminal Deadline Violation

Require: Finish times F **Ensure:** Terminal deadline violation V_{dead}

```
1:  $V_{\text{dead}} \leftarrow 0$ 
2: for each terminal job  $j$  with absolute deadline  $d_j$  do
3:   if  $F_j > d_j$  then
4:      $V_{\text{dead}} \leftarrow V_{\text{dead}} + F_j - d_j$ 
5:   end if
6: end for
7: return  $V_{\text{dead}}$ 
```

Algorithm 13 Same-Core Overlap Violation

Require: Start times S , finish times F , job-to-core assignment A^J **Ensure:** Same-core overlap violation V_{overlap}

```
1: for each core  $c \in C$  do
2:    $I_c \leftarrow []$ 
3: end for
4: for each job  $j \in J$  do
5:    $c \leftarrow A^J_j$ 
6:   Append  $(S_j, F_j, j)$  to  $I_c$ 
7: end for
8:  $V_{\text{overlap}} \leftarrow 0$ 
9: for each core  $c \in C$  do
10:  Sort  $I_c$  by ascending start time, finish time, and job index
11:   $f_{\text{prev}} \leftarrow \emptyset$ 
12:  for each interval  $(s, f, j) \in I_c$  do
13:    if  $f_{\text{prev}} \neq \emptyset$  and  $s < f_{\text{prev}}$  then
14:       $V_{\text{overlap}} \leftarrow V_{\text{overlap}} + f_{\text{prev}} - s$ 
15:    end if
16:    if  $f_{\text{prev}} = \emptyset$  or  $f > f_{\text{prev}}$  then
17:       $f_{\text{prev}} \leftarrow f$ 
18:    end if
19:  end for
20: end for
21: return  $V_{\text{overlap}}$ 
```

Algorithm 14 Jitter Violation

Require: Start times S , maximum allowed jitter Δ **Ensure:** Jitter violation V_{jitter}

```
1: if  $\Delta$  is not defined or  $\Delta \leq 0$  then
2:   return 0
3: end if
4:  $V_{\text{jitter}} \leftarrow 0$ 
5: for each chain-root job  $j \in J$  do
6:    $s_j^{\text{max}} \leftarrow r_j + \Delta$ 
7:   if  $S_j > s_j^{\text{max}}$  then
8:      $V_{\text{jitter}} \leftarrow V_{\text{jitter}} + S_j - s_j^{\text{max}}$ 
9:   end if
10: end for
11: return  $V_{\text{jitter}}$ 
```
